МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ

по лабораторной работе №4

по дисциплине «Вычислительная математика»

Тема: Решение нелинейных уравнений. Метод хорд.

Студент гр. 4342 Кринкин Н.К.

Преподаватель Пуеров Г.Ю.

Санкт-Петербург

2026

Цель работы.

Вариант 8. $f(x) = 2x^2 - x^4 - 1 + ln(x)$

Изучить метод хорд для решения нелинейных уравнений. Найти корень заданного уравнения с заданной точностью, исследовать зависимость количества итераций от требуемой точности и оценить устойчивость метода к ошибкам в исходных данных.

Задание.

В лабораторной работе №4 предлагается, используя программы - функции HORDA и Round из файла methods.cpp (файл заголовков methods.h), найти корень уравнения $f(x) = 0$ методом хорд с заданной точностью Eps, исследовать скорость сходимости и обусловленности метода.

Порядок выполнения работы должен быть следующим:

1) Графически или аналитически отделить корень уравнения $f(x) = 0$ (т.е. найти отрезки [Left, Right], на которых функция $f(x)$ удовлетворяет условиям теоремы Коши).

2) Составить подпрограмму - функцию вычисления функции $f(x)$, предусмотрев округление значений функции с заданной точностью Delta с использованием программы Round.

3) Составить головную программу, вычисляющую корень уравнения $f(x) = 0$ и содержащую обращение к подпрограмме f(x), HORDA, Round и индикацию результатов.

4) Провести вычисления по программе. Теоретически и экспериментально исследовать скорость сходимости и обусловленность метода.

Основные теоретические положения.

Пусть найден отрезок [a, b], на котором функция $f(x)$ меняет знак. Для определенности положим f (a)>0, f (b)<0. В методе хорд процесс итераций состоит в том, что в качестве приближений к корню уравнения $f(x) = 0$ принимаются значения c0, c1, . . . точек пересечения хорды с осью абсцисс, как это показано на рис.1.

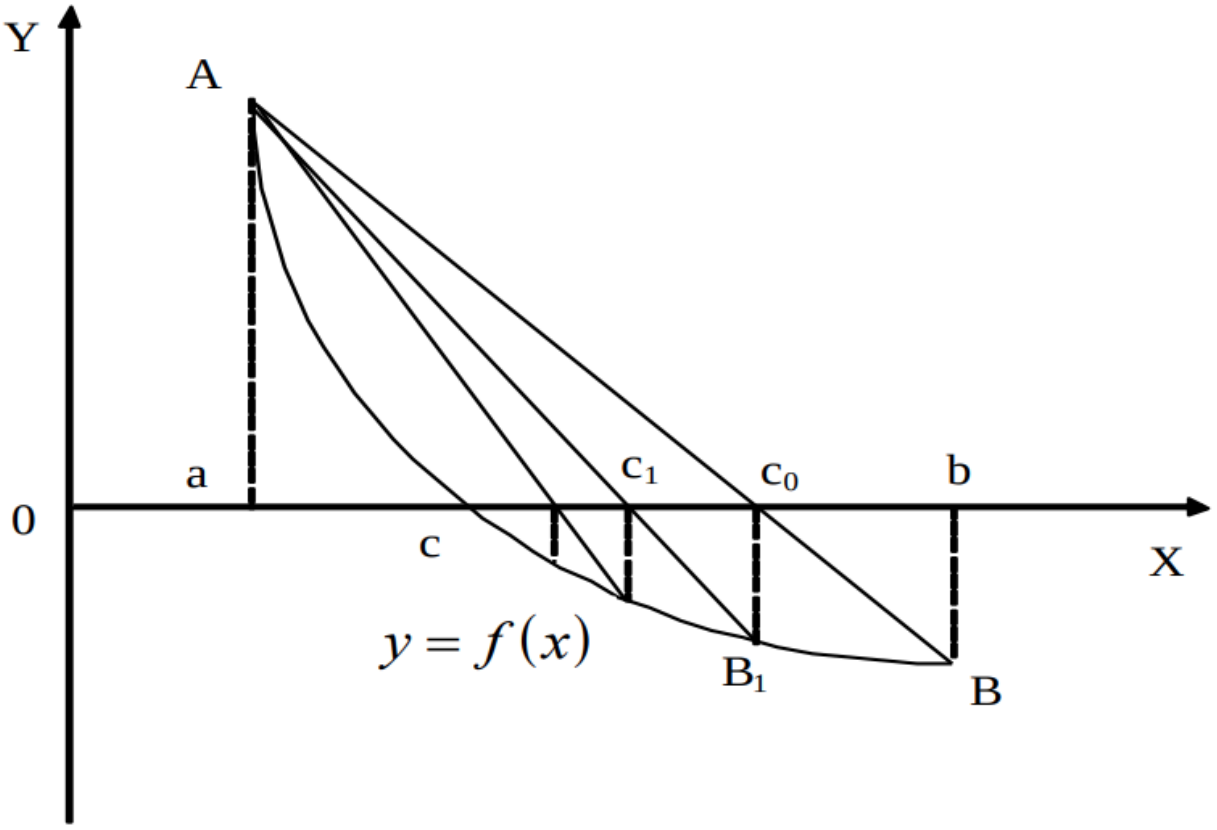

Рис. 1

Сначала находится уравнение хорды AB:

$$\frac{y - f(a)}{f(b) - f(a)} = \frac{x - a}{b - a}$$

Для точки пересечения ее с осью абсцисс (x=c0, y=0) получается уравнение:

$$c_0 = a - \frac{b-a}{f(b)-f(a)} f(a)$$

Далее сравниваются знаки величин $f(a)$ и $f(c)$ и для рассматриваемого случая оказывается, что корень находится в интервале (a, c0), так как $f(a)f(c0) < 0$. Отрезок [c0,b] отбрасывается. Следующая итерации состоит в определении нового приближения c1 как точки пересечения хорды AB_1 с осью абсцисс и т.д. Итерационный процесс продолжается до тех пор, пока значение $f(c_n)$ не станет по модулю меньше заданного числа.

Алгоритмы методов бисекции и хорд похожи, однако метод хорд в ряде случаев дает более быструю сходимость итерационного процесса, причем успех его применения, как и метода бисекции, гарантирован.

Выполнение работы.

1) Нахождение отрезка

   Найдем отрезок, на котором функция меняет свой знак. Для этого найдем производную и определим точки экстремумов:

   $$f(x) = 2x^2 - x^4 - 1 + ln(x) \qquad (*)\ x > 0$$

   $$f'(x) = 4x - 4x^3 - 1/x$$

   $$f'(x) = (2x^2 - 1)^2/x$$

   $x = 1/\sqrt{2}$ – в промежутке x > 0

   Поэтому возьмем точку a > $1/\sqrt{2}$, в данной работе a = 0.85. Точка b = 1.15. Выбранный отрезок также можно дополнительно проверить при помощи графика (рис. 2)

2) Реализуем функцию f(x), используя инструменты стандартной библиотеки C++:

```
double F(double x) {
    return 2 * pow(x, 2) - pow(x, 4) - 1 - log(x);
}
```

   Полный код программы представлен в приложении А.

Рисунок 1 - график функции $f(x) = 2x^2 - x^4 - 1 + ln(x)$

3) В основной файле определим функцию main, которая представляет, которая представляет из себя цикл, перебирающий значение порядка точности ε и выводит результат работы функции HORDA с точностью delta

4) Проведены вычисления, построен график (Рисунок 3).

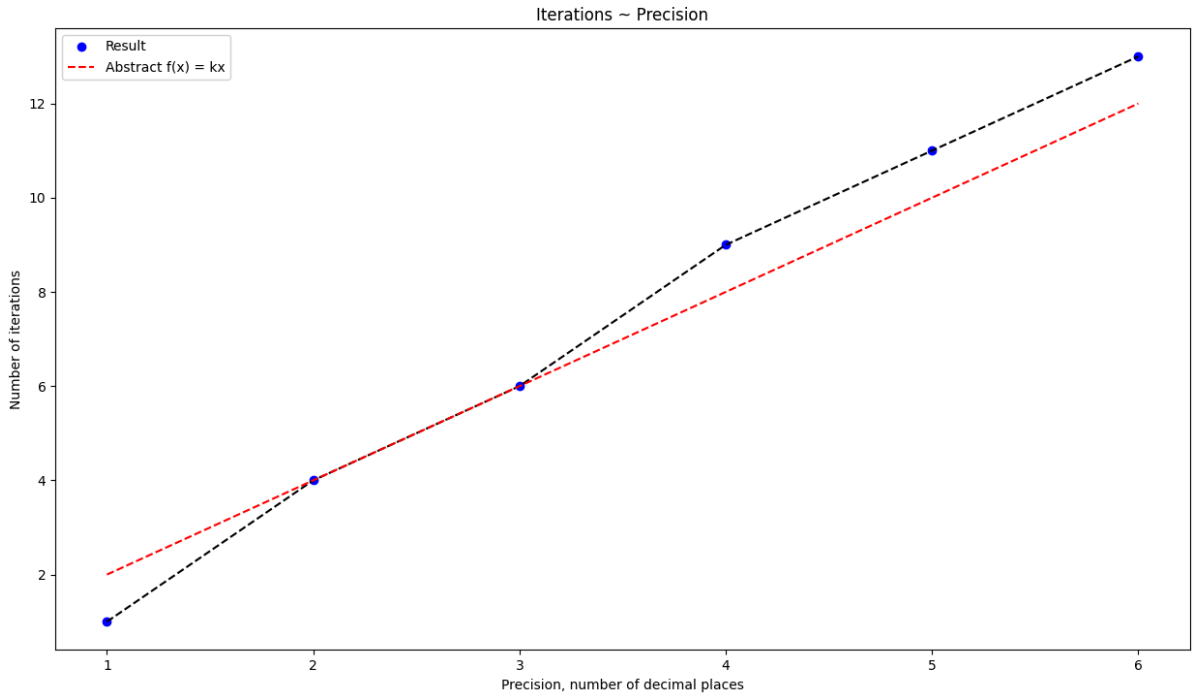

Рисунок 2. График зависимости количества итераций от порядка точности

Если провести исследование графика, можно понять, что количество итераций растет линейно относительно порядка точности ε (количества знаков после запятой).

При непосредственная зависимость числа итераций от точности ε будет логарифмической (рисунок 3): с ростом числа итераций точность вычислений растет экспоненциально и наоборот, чем меньшую допустимую погрешность (значение ε) мы задаем, тем меньше итераций нужно.

5) Для исследования чувствительности метода ко входным данным была написана дополнительная программа. Программа производит итерацию точности delta (от 0.1 до 10^-6) и применяет алгоритм из пункта 3. Результаты работы метода бисекции при различных точностях delta представлены в ниже:

```
For Delta = 0.100000:
     For epsilon = 0.100000: x = 0.900000, iter = 1
     For epsilon = 0.010000: x = 1.000000, iter = 4
     For epsilon = 0.001000: x = 1.000000, iter = 6
     For epsilon = 0.000100: x = 1.000000, iter = 9
```

```
     For epsilon = 0.000010: x = 1.000000, iter = 11
     For epsilon = 0.000001: x = 1.000000, iter = 13
For Delta = 0.010000:
     For epsilon = 0.100000: x = 0.930000, iter = 1
     For epsilon = 0.010000: x = 1.000000, iter = 4
     For epsilon = 0.001000: x = 1.000000, iter = 6
     For epsilon = 0.000100: x = 1.000000, iter = 9
     For epsilon = 0.000010: x = 1.000000, iter = 11
     For epsilon = 0.000001: x = 1.000000, iter = 13
For Delta = 0.001000:
     For epsilon = 0.100000: x = 0.928000, iter = 1
     For epsilon = 0.010000: x = 0.995000, iter = 4
     For epsilon = 0.001000: x = 0.999000, iter = 6
     For epsilon = 0.000100: x = 1.000000, iter = 9
     For epsilon = 0.000010: x = 1.000000, iter = 11
     For epsilon = 0.000001: x = 1.000000, iter = 13
For Delta = 0.000100:
     For epsilon = 0.100000: x = 0.927900, iter = 1
     For epsilon = 0.010000: x = 0.995100, iter = 4
     For epsilon = 0.001000: x = 0.999300, iter = 6
     For epsilon = 0.000100: x = 1.000000, iter = 9
     For epsilon = 0.000010: x = 1.000000, iter = 11
     For epsilon = 0.000001: x = 1.000000, iter = 13
For Delta = 0.000010:
     For epsilon = 0.100000: x = 0.927910, iter = 1
     For epsilon = 0.010000: x = 0.995120, iter = 4
     For epsilon = 0.001000: x = 0.999270, iter = 6
     For epsilon = 0.000100: x = 0.999960, iter = 9
     For epsilon = 0.000010: x = 0.999990, iter = 11
     For epsilon = 0.000001: x = 1.000000, iter = 13
For Delta = 0.000001:
     For epsilon = 0.100000: x = 0.927909, iter = 1
     For epsilon = 0.010000: x = 0.995121, iter = 4
     For epsilon = 0.001000: x = 0.999271, iter = 6
     For epsilon = 0.000100: x = 0.999958, iter = 9
     For epsilon = 0.000010: x = 0.999994, iter = 11
     For epsilon = 0.000001: x = 0.999999, iter = 13
```

Исходя из результатов исследования, можно сделать вывод об устойчивости метода отрезков к ошибкам в округлении.

Выводы.

В ходе работы был изучен метод бисекции (половинного деления). Для уравнения $f(x) = 2x^2 - x^4 - 1 + ln(x)$ корень был успешно найден на отрезке [0.85; 1.15] с точностью от 0.1 до 10^-6. Построенный график показал линейную зависимость числа итераций от порядка точности: увеличение

точности (уменьшение ε) требует ожидаемого роста количества вычислений, что соответствует теории. Исследование обусловленности показало, что метод хорд является устойчивым к малым ошибкам округления значений функции, однако при грубых возмущениях (большая Delta) процесс может замедлиться или привести к потере точности нахождения корня.

ПРИЛОЖЕНИЕ А
ИСХОДНЫЙ КОД ПРОГРАММЫ

Файл `precision.cpp`:

```
#include <iostream>
#include <cstdio>
#include <cmath>
#include <string>
#include <vector>

#include "methods.h"

double F(double x) {
    return 2 * pow(x, 2) - pow(x, 4) - 1 - log(x);
}

int main(int argc, char *argv[]) {
    int n;
    double x;
    double eps;
    double delta = pow(10, -4);

    FILE *out_file = fopen(argv[1], "w");

    double a = 0.85;
    double b = 1.15;

    for (int i=1; i<7; i++) {
        eps = pow(10, -i);
        x = HORDA(a, b, eps, n);
        fprintf(out_file, "%d %d\n", i, n);
    }
    fclose(out_file);

    return 0;
}
```

Файл `lab.cpp`:

```
#include <iostream>
#include <cstdio>
#include <cmath>
#include <string>
#include <vector>

#include "methods.h"

using namespace std;


double F(double x) {
    return 2 * pow(x, 2) - pow(x, 4) - 1 - log(x);
}
```

```
int main() {
    double x;
    double eps;
    double delta;

    int it;
    cout << "Enter epsilon presicion (decimal points): ";
    cin >> it; if (it <= 0) it = 1;
    eps = pow(10, -it);

    cout << "Enter delta presicion (decimal points): ";
    cin >> it; if (it <= 0) it = 1;
    delta = pow(10, -it);

    double a = 0.1;
    double b = 1.5;

    int n = 0;
    x = HORDA(a, b, eps, n);

    printf("\tFor epsilon = %lf: x = %lf with delta = %lf, iter =
%d\n", eps, Round(x, delta), delta, n);

    return 0;
}
```

Файл `input_errors.cpp`:

```
#include <iostream>
#include <cstdio>
#include <cmath>
#include <string>
#include <vector>

#include "methods.h"

double F(double x) {
    return 2 * pow(x, 2) - pow(x, 4) - 1 - log(x);
}

int main() {
    int n;
    double x;
    double eps;
    double delta;

    double a = 0.1;
    double b = 1.5;

    for (int di=1; di<7; di++) {
        delta = pow(10, -di);
        printf("For Delta = %lf:\n", delta);

        for (int i=1; i<7; i++) {
            eps = pow(10, -i);
            x = HORDA(a, b, eps, n);
```

```
                    printf("\tFor epsilon = %lf: x = %lf, iter = %d\n",
eps, Round(x, delta), n);
                }
            }

            return 0;
        }
```

Файл `graph.py`:

```
import matplotlib
matplotlib.use("TkAgg")

import os
import sys
from matplotlib import pyplot as plt

data_x = []
data_y = []

if not os.path.exists(sys.argv[1]):
    raise FileNotFoundError(f"No {sys.argv[1]} file!")
file = open(sys.argv[1], "r")

for line in file.readlines():
    if line:
        x, y = line.split()
        data_x.append(int(x))
        data_y.append(int(y))

plt.plot(data_x, data_y, "bo", label="Result")
plt.plot(data_x, data_y, "k--")
plt.plot(data_x,   list(map(lambda   x:   3*x,   data_x)),   "r--",
label="Abstract f(x) = kx")
plt.title("Iterations ~ Precision")
plt.xlabel("Precision, number of decimal places")
plt.ylabel("Number of iterations")
plt.legend()
plt.show()
```

Файл `Makefile`:

```
SRC_DIR = ${CURDIR}/src
WORK_DIR = ${SRC_DIR}/workdir
MAT_DIR = ${SRC_DIR}/Materials

lab: ${WORK_DIR}/lab
${WORK_DIR}/lab

precision: ${WORK_DIR}/precision.txt ${WORK_DIR}/graph.py
python3 ${WORK_DIR}/graph.py ${WORK_DIR}/precision.txt

input_errors: ${WORK_DIR}/input_errors.o
${WORK_DIR}/input_errors.o

${WORK_DIR}/precision.txt: ${WORK_DIR}/precision.o
${WORK_DIR}/precision.o ${WORK_DIR}/precision.txt
```

```
	${WORK_DIR}/lab: ${WORK_DIR}/lab.cpp ${MAT_DIR}
	g++ -g -I${MAT_DIR} ${WORK_DIR}/lab.cpp -o ${WORK_DIR}/lab

	${WORK_DIR}/precision.o: ${WORK_DIR}/precision.cpp ${MAT_DIR}
	g++	-g	-I${MAT_DIR}	${WORK_DIR}/precision.cpp	-o
${WORK_DIR}/precision.o

	${WORK_DIR}/input_errors.o:		${WORK_DIR}/input_errors.cpp
${MAT_DIR}
	g++	-g	-I${MAT_DIR}	${WORK_DIR}/input_errors.cpp	-o
${WORK_DIR}/input_errors.o

	help:
	@echo "Available options:"
	@echo "	make [lab] -- calculate f(x) = 0 and show results"
	@echo " make precision -- calculate f(x) for different epsilon for
given delta"
	@echo "	make input_errors -- show input data sensitivity"
```